TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



HỌ VÀ TÊN SINH VIÊN 1 - MSSV
HỌ VÀ TÊN SINH VIÊN 2 - MSSV

CHỌN VÍ DỤ CÓ TÍNH ĐẾN NHÃN CHO MÔ HÌNH NGÔN NGỮ LỚN TRONG PHÂN LOẠI TẤN CÔNG WEB

BÁO CÁO GIỮA KỲ MÔN DỰ ÁN CÔNG NGHỆ THÔNG TIN

MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



HỌ VÀ TÊN SINH VIÊN 1 - MSSV
HỌ VÀ TÊN SINH VIÊN 2 - MSSV

CHỌN VÍ DỤ CÓ TÍNH ĐẾN NHÃN CHO MÔ HÌNH NGÔN NGỮ LỚN TRONG PHÂN LOẠI TẤN CÔNG WEB

BÁO CÁO GIỮA KỲ MÔN DỰ ÁN CÔNG NGHỆ THÔNG TIN

MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG DỮ LIỆU

Người hướng dẫn

[Học hàm, học vị và họ tên giảng viên hướng dẫn]

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn [Học hàm, học vị và họ tên giảng viên hướng dẫn] đã định hướng đề tài, góp ý về phương pháp và theo dõi tiến độ của nhóm trong học phần Dự án Công nghệ Thông tin. Những câu hỏi của thầy/cô về dữ liệu và cách đánh giá đã giúp nhóm nhìn lại các giả định và làm phần thực nghiệm cẩn thận hơn.

Chúng em cũng xin cảm ơn Khoa Công nghệ thông tin, Trường Đại học Tôn Đức Thắng đã tạo điều kiện về chương trình học và tài liệu, và cảm ơn các tác giả bộ dữ liệu SR-BH 2020 đã công bố dữ liệu với giấy phép mở.

Đây là báo cáo giữa kỳ nên còn nhiều phần chưa hoàn thiện. Chúng em mong nhận được nhận xét và góp ý của thầy/cô để hoàn thiện đề tài ở giai đoạn cuối kỳ.

TP. Hồ Chí Minh, ngày ... tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng tôi xin cam đoan đây là công trình nghiên cứu của riêng chúng tôi và được sự hướng dẫn khoa học của [Học hàm, học vị và họ tên giảng viên hướng dẫn]. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào chúng tôi xin hoàn toàn chịu trách nhiệm về nội dung báo cáo của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ... tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

CHỌN VÍ DỤ CÓ TÍNH ĐẾN NHÃN CHO MÔ HÌNH NGÔN NGỮ LỚN TRONG PHÂN LOẠI TẤN CÔNG WEB

TÓM TẮT

Đề tài nghiên cứu cách chọn ví dụ có nhãn (demonstrations) cho mô hình ngôn ngữ lớn (LLM) khi phân loại HTTP request thành sáu lớp: Normal, SQL Injection, Path Traversal, Code Injection, HTTP Response Splitting và OS Command Injection, theo hướng chỉ suy luận, không huấn luyện mô hình. Phương pháp đề xuất lấy top-N ứng viên giống request nhất rồi cộng điểm cho ứng viên có nhãn trùng với một dự đoán sơ bộ (prior) lấy từ LLM zero-shot hoặc kNN. Trên dữ liệu SR-BH 2020 (907.815 bản ghi, còn 466.267 request duy nhất sau tiền xử lý), nhóm đã chia tập theo nhóm request gần trùng, không có request nào trùng giữa các tập, và cài đặt toàn bộ hệ thống với 55 bài kiểm thử tự động. Kết quả giữa kỳ trên 200 request validation cho thấy 23,0% request có ví dụ giống nhất mang nhãn khác, riêng lớp Normal là 45,5%. kNN vote không dùng LLM đạt Macro-F1 0,800, còn LLM zero-shot (gpt-oss-20b) đạt 0,835. Xếp hạng lại theo prior kNN nâng tỷ lệ ví dụ cùng nhãn từ 0,726 lên 0,809, nhưng làm 16,5% request không còn ví dụ đúng lớp nào khi prior sai. Các pipeline có ví dụ đang được chạy trong giới hạn quota API miễn phí và sẽ hoàn thành ở cuối kỳ.

LABEL-AWARE RETRIEVAL FOR LLM-BASED WEB ATTACK DETECTION

ABSTRACT

This project studies how to select labeled demonstrations for a large language model (LLM) that classifies HTTP requests into six classes without any training. The proposed method retrieves the top-N most similar labeled requests and boosts candidates whose label matches a preliminary prediction (prior) from zero-shot LLM inference or kNN voting. On the SR-BH 2020 dataset (907,815 records; 466,267 unique requests after preprocessing) the data were split by groups of near-duplicate requests with no overlap between splits, and the full system was implemented with 55 automated tests. On 200 validation requests, 23.0% of queries have a nearest neighbour with a different label (45.5% for Normal). kNN voting without an LLM reaches a macro-F1 of 0.800, and zero-shot gpt-oss-20b reaches 0.835. Prior-based reranking raises the share of same-label demonstrations from 0.726 to 0.809, but leaves 16.5% of queries without any correct-class example when the prior is wrong. Few-shot pipelines are being run within the free API quota for the final report.

MỤC LỤC

DANH MỤC HÌNH VẼ vii

DANH MỤC BẢNG BIỂU viii

DANH MỤC CÁC CHỮ VIẾT TẮT ix

CHƯƠNG 1. MỞ ĐẦU 1

1.1 Đặt vấn đề 1

1.2 Mục tiêu và câu hỏi nghiên cứu 2

1.3 Phạm vi và đóng góp dự kiến 2

1.4 Bố cục báo cáo 3

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN 4

2.1 HTTP request và các loại tấn công trong phạm vi 4

2.2 Mô hình ngôn ngữ lớn và học trong ngữ cảnh 4

2.3 Embedding và tìm kiếm theo độ tương đồng 4

2.4 Các nghiên cứu về chọn ví dụ cho ICL 5

2.5 Chỉ số đánh giá và kiểm định 5

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ 7

3.1 Nguồn dữ liệu 7

3.2 Tiền xử lý 7

3.3 Chia tập và kiểm tra rò rỉ dữ liệu 8

3.4 Chất lượng nhãn 8

CHƯƠNG 4. PHƯƠNG PHÁP 10

4.1 Tổng quan hệ thống 10

4.2 Biểu diễn request cho embedding 10

4.3 Truy hồi ứng viên 10
4.4 Các chiến lược chọn ví dụ 11
4.5 Prompt và cách đọc câu trả lời 11
4.6 Quy trình đánh giá 12
CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ 13
5.1 Cấu trúc mã nguồn 13
5.2 Kiểm thử 13
5.3 Các sự cố đã xử lý 14
CHƯƠNG 6. KẾT QUẢ GIỮA KỲ 15
6.1 RQ1: ví dụ giống nhất có khác nhãn không 15
6.2 Đối chứng kNN không dùng LLM 15
6.3 Ảnh hưởng của prior lên các ví dụ được chọn 16
6.4 Kết quả LLM zero-shot (A0) 18
6.5 Giới hạn quota và điều chỉnh quy mô thí nghiệm 19
CHƯƠNG 7. KẾT LUẬN GIỮA KỲ VÀ KẾ HOẠCH 21
7.1 Kết quả đạt được 21
7.2 Hạn chế 21
7.3 Kế hoạch giai đoạn cuối kỳ 22
TÀI LIỆU THAM KHẢO 23
PHỤ LỤC A. PHÂN CÔNG CÔNG VIỆC 24
PHỤ LỤC B. LỆNH TÁI LẬP 25

DANH MỤC HÌNH VẼ

Hình 1.1: Cặp request Normal và SQL Injection gần giống hệt nhau trong SR-BH 2020 2

Hình 3.1: Phân bố 14 nhãn gốc của SR-BH 2020 trên trục logarit 7

Hình 4.1: Kiến trúc hệ thống và vị trí của prior trong Pipeline C 10

Hình 6.1: (a) Tỷ lệ top-1 khác nhãn theo lớp; (b) LCR@4 theo lớp của các chiến lược chọn ví dụ 15

Hình 6.2: Ma trận nhầm lẫn của kNN vote (k = 5) trên validation 16

Hình 6.3: Ma trận nhầm lẫn của A0 trên validation 19

DANH MỤC BẢNG BIỂU

Bảng 1.1: Ba câu hỏi nghiên cứu và cách trả lời.....2

Bảng 2.1: Sáu lớp trong phạm vi đề tài.....4

Bảng 2.2: Các nghiên cứu liên quan 5

Bảng 3.1: Các bước tiền xử lý và số lượng.....8

Bảng 3.2: Kích thước các tập và tập con dùng trong thực nghiệm.....8

Bảng 4.1: Các pipeline và cách chọn k ví dụ 11

Bảng 5.1: Các module chính 13

Bảng 5.2: Các tính chất được kiểm thử tự động 14

Bảng 5.3: Sự cố kỹ thuật và cách xử lý.....14

Bảng 6.1: Kết quả kNN vote trên validation (200 request) 16

Bảng 6.2: LCR@4 của các chiến lược chọn ví dụ trên validation.....17

Bảng 6.3: Quét λ cho C với prior kNN (k = 4, validation) 17

Bảng 6.4: So sánh LLM zero-shot (A0) với kNN vote (k = 5) trên validation 18

Bảng 7.1: Kế hoạch đến khi nộp báo cáo cuối kỳ.....22

Bảng A.1: Phân công và mức độ hoàn thành 24

DANH MỤC CÁC CHỮ VIẾT TẮT

API	Application Programming Interface, giao diện lập trình ứng dụng
CAPEC	Common Attack Pattern Enumeration and Classification, danh mục mẫu tấn công
CI	Confidence Interval, khoảng tin cậy
FPR	False Positive Rate, tỷ lệ báo động nhầm
HTTP	Hypertext Transfer Protocol
ICL	In-Context Learning, học trong ngữ cảnh
kNN	k-Nearest Neighbors, k láng giềng gần nhất
LCR@k	Label Consistency Rate at k, tỷ lệ ví dụ cùng nhãn trong top-k
LLM	Large Language Model, mô hình ngôn ngữ lớn
RAG	Retrieval-Augmented Generation, sinh văn bản có truy hồi
RQ	Research Question, câu hỏi nghiên cứu
SQLi	SQL Injection, chèn câu lệnh SQL
WAF	Web Application Firewall, tường lửa ứng dụng web

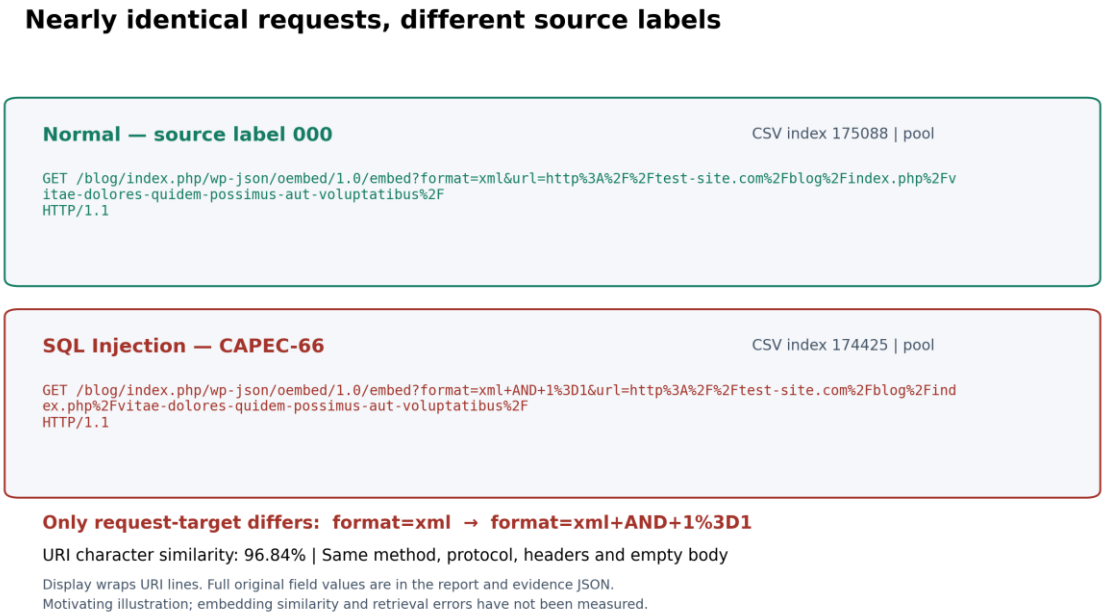
CHƯƠNG 1. MỞ ĐẦU

1.1 Đặt vấn đề

Mỗi thao tác trên một website, từ tìm kiếm sản phẩm đến đăng nhập, tạo ra một HTTP request gửi đến máy chủ. Kẻ tấn công lợi dụng chính các request này để chèn câu lệnh SQL, đường dẫn truy cập file trái phép hay lệnh hệ điều hành. Vì vậy, đọc nội dung request là một cách phát hiện dấu hiệu tấn công web. Tường lửa ứng dụng web (WAF) truyền thống như ModSecurity dựa trên luật so khớp mẫu, hiệu quả nhưng phải được viết và bảo trì thủ công.

Mô hình ngôn ngữ lớn (LLM) có thể đọc văn bản bán cấu trúc và phân loại theo hướng dẫn mà không cần huấn luyện lại, nhờ kỹ thuật học trong ngữ cảnh (In-Context Learning, ICL): đưa vào prompt một vài ví dụ đã có nhãn. Chất lượng dự đoán phụ thuộc mạnh vào các ví dụ này [1]. Cách phổ biến là chọn ví dụ giống request nhất theo embedding. Tuy nhiên, trong miền HTTP, hai request có thể giống gần như hoàn toàn mà mang nhãn khác nhau.

Hình 1.1 là một cặp request thật trong SR-BH 2020: giống hệt nhau về method, protocol, 11 header và body, chỉ khác chuỗi `+AND+1%3D1` trong tham số. Độ tương đồng ký tự của toàn request là 99,50%, nhưng một request mang nhãn Normal, request kia là SQL Injection. Nếu ví dụ giống nhất lại mang nhãn khác, LLM có thể bị dẫn sai. Đây là động cơ của đề tài.



Hình 1.1: Cặp request Normal và SQL Injection gần giống hệt nhau trong SR-BH 2020

1.2 Mục tiêu và câu hỏi nghiên cứu

Đề tài kiểm chứng việc dùng thông tin nhãn khi chọn ví dụ có giúp LLM phân loại tốt hơn so với chỉ chọn theo độ tương đồng hay không. Nhóm đặt ba câu hỏi nghiên cứu (Bảng 1.1) trước khi chạy thực nghiệm.

Bảng 1.1: Ba câu hỏi nghiên cứu và cách trả lời

Câu hỏi	Cách trả lời	Trạng thái
RQ1. Semantic retrieval thường xuyên lấy về ví dụ khác nhãn với request cần phân loại đến mức nào?	Tỷ lệ top-1 khác nhãn, LCR@k; không cần LLM	Đã có kết quả (Chương 6)
RQ2. Label-aware reranking (C) có cải thiện Macro-F1 so với semantic retrieval (B) và đối chứng cân bằng lớp (BAL) không?	So sánh trên cùng tập query, bootstrap và McNemar	A0 xong; B, BAL, C đang chạy
RQ3. C giúp hay gây hại trong trường hợp nào: prior đúng, prior sai, lớp ít mẫu?	Bảng prior đúng/sai, C-oracle, F1 từng lớp	Đã có phân tích sơ bộ trên retrieval

1.3 Phạm vi và đóng góp dự kiến

Đề tài dùng một bộ dữ liệu (SR-BH 2020) với sáu lớp đơn nhãn, đánh giá offline, không huấn luyện hay tinh chỉnh mô hình và không xây WAF chạy thời gian

thực. Hệ thống chỉ phân loại dựa trên nội dung request, không xác minh tấn công có khai thác thành công hay không.

Đóng góp dự kiến gồm ba phần. Thứ nhất, một pipeline chỉ suy luận có thể tái lập, so sánh công bằng năm cách chọn ví dụ trên cùng LLM, prompt và tập đánh giá. Thứ hai, số liệu định lượng về hiện tượng giống nhưng khác nhãn trong dữ liệu HTTP. Thứ ba, phân tích khi nào thông tin nhãn giúp hoặc gây hại, gồm cả mức trần khi biết nhãn thật (C-oracle). Ý tưởng dùng nhãn khi chọn ví dụ đã có trong xử lý ngôn ngữ tự nhiên [3]; nhóm không khẳng định đây là phương pháp đầu tiên.

1.4 Bố cục báo cáo

Chương 2 trình bày cơ sở lý thuyết và các nghiên cứu liên quan. Chương 3 mô tả dữ liệu và tiền xử lý. Chương 4 trình bày phương pháp. Chương 5 mô tả cài đặt và kiểm thử. Chương 6 báo cáo kết quả giữa kỳ. Chương 7 tổng kết và nêu kế hoạch cuối kỳ.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ NGHIÊN CỨU LIÊN QUAN

2.1 HTTP request và các loại tấn công trong phạm vi

Một HTTP request gồm dòng yêu cầu (method, đường dẫn kèm tham số, phiên bản giao thức), các header và phần thân (body). Payload tấn công thường nằm trong tham số của URI hoặc trong body, đôi khi trong header như User-Agent. Bảng 2.1 tóm tắt sáu lớp đề tài phân loại, theo danh mục CAPEC mà bộ dữ liệu sử dụng.

Bảng 2.1: Sáu lớp trong phạm vi đề tài

Lớp (mã CAPEC)	Dấu hiệu điển hình
Normal (000)	Request bình thường, không có payload tấn công trong phạm vi
SQL Injection (66)	Chèn cú pháp SQL vào tham số, ví dụ AND 1=1, UNION SELECT, SLEEP()
Path Traversal (126)	Chuỗi ../ hoặc dạng mã hóa để truy cập file ngoài thư mục cho phép
Code Injection (242)	Chèn mã của ngôn ngữ phía máy chủ để thực thi, ví dụ print(chr(...))
HTTP Response Splitting (34)	Chèn ký tự CR/LF (%0d%0a) và header giả để tách phản hồi HTTP
OS Command Injection (88)	Chèn lệnh hệ điều hành, ví dụ ;cat /etc/passwd, Shellshock trong header

2.2 Mô hình ngôn ngữ lớn và học trong ngữ cảnh

ICL cho LLM thực hiện nhiệm vụ bằng cách đưa vào prompt phần hướng dẫn và một số ví dụ có lời giải. Trọng số mô hình không thay đổi; các ví dụ chỉ có tác dụng trong lần gọi đó. Không có ví dụ gọi là zero-shot, có k ví dụ gọi là k-shot. Vì không cần huấn luyện, ICL phù hợp khi muốn thay đổi bộ ví dụ hoặc tập nhãn mà không xây lại mô hình, đổi lại mỗi lần phân loại tốn một lượt gọi LLM.

2.3 Embedding và tìm kiếm theo độ tương đồng

Mô hình embedding biến mỗi đoạn văn bản thành một vector số sao cho văn bản giống nhau có vector gần nhau. Đề tài dùng all-MiniLM-L6-v2 thuộc họ Sentence-BERT [7], cho vector 384 chiều. Độ tương đồng giữa hai request là cosine

của góc giữa hai vector; khi vector đã chuẩn hóa độ dài 1, cosine chính là tích vô hướng.

$$\mathrm{sim}(q, d) = q \cdot d, \quad \|q\| = \|d\| = 1 \tag{2.1}$$

Thuật toán k láng giềng gần nhất (kNN) dự đoán nhãn bằng cách lấy k ví dụ có độ tương đồng cao nhất và chọn nhãn chiếm ưu thế. Đề tài dùng kNN vừa làm đối chứng không dùng LLM, vừa làm nguồn prior rẻ cho phương pháp đề xuất.

2.4 Các nghiên cứu về chọn ví dụ cho ICL

Bảng 2.2 tóm tắt các công trình gần với đề tài. Điểm chung là ví dụ được chọn ảnh hưởng mạnh đến kết quả ICL; điểm khác của đề tài là miền dữ liệu HTTP, nơi các request rất giống nhau về hình thức.

Bảng 2.2: Các nghiên cứu liên quan

Công trình	Ý tưởng chính	Liên hệ với đề tài
KATE [1]	Chọn ví dụ gần nhất theo embedding thường tốt hơn chọn ngẫu nhiên	Cơ sở của Pipeline B
EPR [2]	Huấn luyện bộ retriever để chọn ví dụ hữu ích	Có huấn luyện; đề tài không dùng
CLL-RetICL [3]	Dùng thông tin ngữ nghĩa của nhãn khi truy hồi ví dụ cho phân loại văn bản	Gần nhất về ý tưởng; đề tài dùng prior dự đoán thay cho embedding nhãn
ICL with Noisy Labels [4]	Ảnh hưởng của nhãn sai trong ví dụ đến ICL	SR-BH có dấu hiệu nhãn nhiễu
DemoShapley [5]	Định giá đóng góp của từng ví dụ	Hướng mở rộng, ngoài phạm vi
CyberRAG [6]	RAG kết hợp bộ phân loại để phân loại và báo cáo tấn công web	Cùng miền; đề tài không huấn luyện bộ phân loại riêng

2.5 Chỉ số đánh giá và kiểm định

Precision của một lớp là tỷ lệ request đoán đúng trong số request bị đoán là lớp đó; recall là tỷ lệ request của lớp được đoán ra; F1 là trung bình điều hòa của hai giá trị. Macro-F1 là trung bình F1 của sáu lớp với trọng số bằng nhau, nên lớp ít mẫu không bị lấn át; đây là chỉ số chính của đề tài. Tỷ lệ báo động nhầm (FPR) trên lớp Normal đo phần request bình thường bị báo là tấn công.

LCR@k (Label Consistency Rate) là tỷ lệ ví dụ trong top-k cùng nhãn với request cần phân loại. Chỉ số này chỉ tính được khi đánh giá vì cần nhãn thật, và chỉ

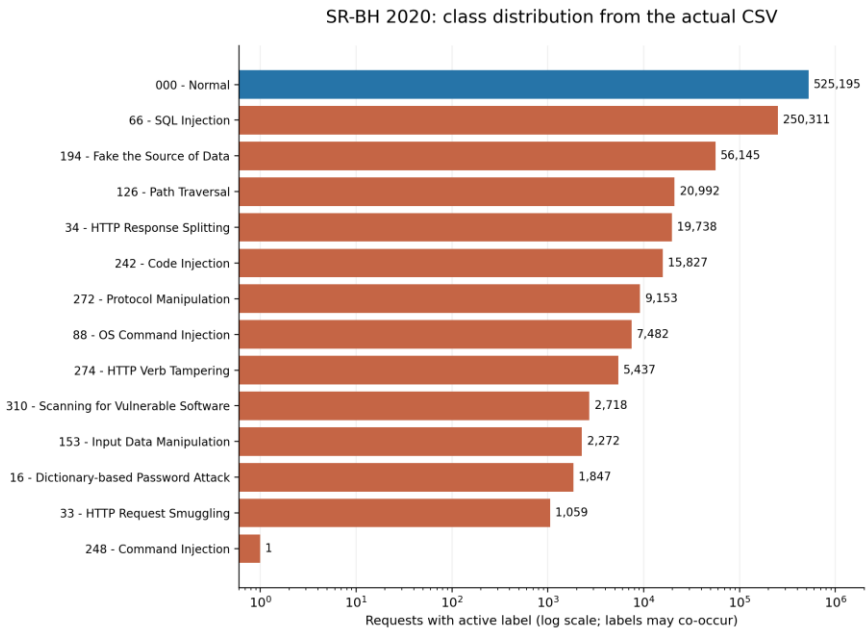
dùng để chẩn đoán: ví dụ cùng nhãn nhiều hơn chưa chắc giúp LLM đúng hơn. Để so sánh hai pipeline trên cùng tập request, đề tài dùng paired bootstrap 2.000 lần cho khoảng tin cậy 95% của chênh lệch Macro-F1 [8] và kiểm định McNemar trên các request mà hai pipeline đúng sai khác nhau [9].

CHƯƠNG 3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1 Nguồn dữ liệu

Đề tài dùng bộ SR-BH 2020 multi-label trên Harvard Dataverse, phiên bản 1.2, giấy phép CC0 1.0 [10]. File data_capec_multilabel.csv dung lượng 436.437.661 byte, mã MD5 khớp với nguồn. Theo mô tả của tác giả, dữ liệu là traffic của một website WordPress trong 12 ngày tháng 7/2020, được ModSecurity ghi nhận và gán nhãn theo CAPEC, có xử lý thủ công hoặc bán tự động.

File gồm 907.815 bản ghi và 38 cột: 5 cột metadata, 15 trường request, 4 trường response và 14 cột nhãn (Normal và 13 lớp CAPEC). Dữ liệu mất cân bằng nặng: Normal có 525.195 bản ghi, SQL Injection 250.311, trong khi lớp nhỏ nhất chỉ có 1 bản ghi (Hình 3.1). Khoảng 97% bản ghi không có body.



Hình 3.1: Phân bố 14 nhãn gốc của SR-BH 2020 trên trục logarit

3.2 Tiền xử lý

Đầu vào của mô hình là chuỗi request_text: một JSON gồm method, URI, protocol, 11 header và body. Payload được giữ nguyên, không giải mã URL và không cắt bớt. Các bước lọc được trình bày ở Bảng 3.1.

Bảng 3.1: Các bước tiền xử lý và số lượng

Bước	Quy tắc	Kết quả
Chọn lớp	Bản ghi có đúng một nhãn trên 14 cột và thuộc sáu lớp	826.884 bản ghi
Loại đa nhãn	Bản ghi có từ hai nhãn trở lên	9.238 bản ghi
Cách ly mâu thuẫn	Request trùng nội dung nhưng khác nhãn	288 request (2.331 bản ghi)
Bỏ trùng	Trùng toàn bộ nội dung thì giữ một	358.720 bản ghi bị bỏ
Kết quả	Request duy nhất còn lại	466.267 request

3.3 Chia tập và kiểm tra rò rỉ dữ liệu

Nếu chia ngẫu nhiên theo dòng, các request cùng khuôn (chỉ khác số thứ tự hay mã phiên) sẽ rơi vào cả kho ví dụ lẫn tập kiểm tra, khiến kết quả đẹp giả tạo. Vì vậy, nhóm che các con số và chuỗi hex dài, băm phần còn lại để gom request thành 447.049 nhóm, rồi chia theo nhóm với tỷ lệ xấp xỉ 70/15/15 và seed 42. Thực nghiệm dùng các tập con cân bằng lớp (Bảng 3.2).

Bảng 3.2: Kích thước các tập và tập con dùng trong thực nghiệm

Tập	Đầy đủ	Tập con	Vai trò
Pool	326.595	3.000 (500/lớp)	Kho ví dụ có nhãn
Development	30	30 (5/lớp)	Chạy thử, sửa lỗi
Validation	69.950	200 (33–34/lớp)	Chọn tham số k, λ, nguồn prior
Test	69.692	600 (100/lớp)	Chỉ chạy một lần sau khi khóa cấu hình

Kiểm tra cho mọi cặp tập đều có **0** request ID, 0 mã băm nội dung và 0 nhóm trùng nhau. Chạy lại toàn bộ tiền xử lý hai lần (một lần trên Linux, một lần trên Windows) cho cùng số lượng và cùng phân bố. Hạn chế còn lại là có 18.328 endpoint (method và đường dẫn) xuất hiện ở cả pool và test, nên kết quả chưa đo khả năng tổng quát sang endpoint mới.

3.4 Chất lượng nhãn

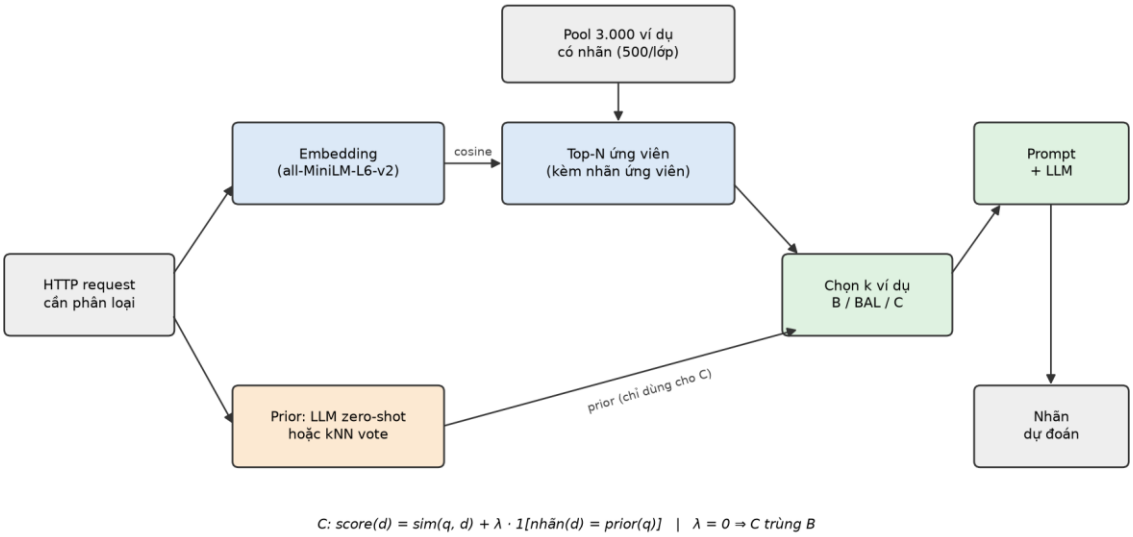
Đọc kỹ 60 request (10 mỗi lớp) từ development và pool cho thấy 5 trong 10 mẫu Normal chứa payload đáng nghi như `sleep(...)`, `cat /etc/passwd` hay `<script>`. Lượt đọc này do công cụ hỗ trợ thực hiện, chưa phải đánh giá độc lập của

hai người, và mẫu quá nhỏ để ước lượng tỷ lệ nhãn sai toàn bộ dữ liệu. Hệ quả là điểm số tính trên nhãn gốc thể hiện mức đồng thuận với nhãn nguồn. Nhóm giữ nguyên nhãn gốc, không tự sửa nhãn, và sẽ cùng đọc lại độc lập 60 mẫu này ở giai đoạn sau.

CHƯƠNG 4. PHƯƠNG PHÁP

4.1 Tổng quan hệ thống

Hình 4.1 mô tả luồng xử lý. Mọi pipeline dùng cùng LLM, cùng khung prompt, cùng pool và cùng số ví dụ k = 4; chỉ cách chọn ví dụ thay đổi. Nhờ vậy chênh lệch kết quả giữa các pipeline đến từ cách chọn ví dụ.



Hình 4.1: Kiến trúc hệ thống và vị trí của prior trong Pipeline C

4.2 Biểu diễn request cho embedding

Trong request_text, các khóa JSON sắp theo thứ tự chữ cái nên 11 header đứng trước URI. Mô hình embedding chỉ đọc 256 token đầu nên có thể cắt mất payload trong URI. Nhóm tạo một cách trình bày khác cho embedding: đặt method, URI, protocol và body lên đầu, các header không rõng ở sau. Nội dung không bị sửa hay giải mã, chỉ đổi thứ tự.

4.3 Truy hồi ứng viên

Mỗi request được mã hóa thành vector 384 chiều đã chuẩn hóa. Với mỗi query, hệ thống tính độ tương đồng với 3.000 ví dụ trong pool bằng một phép nhân ma trận và giữ N = 30 ứng viên giống nhất. Khi hai ứng viên bằng điểm, ứng viên có mã nhỏ

hơn đứng trước để kết quả luôn giống nhau giữa các lần chạy. Kết quả truy hồi chỉ chứa nhãn của ứng viên, không chứa nhãn của query.

4.4 Các chiến lược chọn ví dụ

Bảng 4.1 liệt kê các pipeline. Phương pháp đề xuất C xếp lại top-N theo công thức (4.1).

Bảng 4.1: Các pipeline và cách chọn k ví dụ

Pipeline	Cách chọn ví dụ	Vai trò
kNN vote	Không gọi LLM; đếm nhãn k láng giềng có trọng số theo độ tương đồng	Đối chứng không LLM
A0	Không có ví dụ (zero-shot)	Năng lực ban đầu; nguồn prior
A1	Một bộ 4 ví dụ cố định, chia đều theo lớp (seed 42)	Few-shot không truy hồi (tùy chọn)
B	k ứng viên đầu của top-N	Đối chứng truy hồi chính
BAL	Gom top-N theo lớp, lấy luân phiên mỗi lớp một ví dụ đến đủ k	Kiểm tra lợi ích cân bằng nhãn
C	Xếp lại top-N theo công thức (4.1), lấy k ví dụ điểm cao nhất	Phương pháp đề xuất
C-oracle	Như C nhưng prior là nhãn thật	Chỉ để phân tích mức trần

$$\text{score}(d) = \text{sim}(q, d) + \lambda \cdot 1_{[\text{label}(d)=\text{prior}(q)]} \quad (4.1)$$

Trong đó prior(q) là nhãn đoán trước cho query, lấy từ một trong ba nguồn: dự đoán của A0 (tốn thêm một lượt gọi LLM), kNN vote trên năm láng giềng (không tốn thêm lượt gọi), hoặc nhãn thật trong cấu hình C-oracle chỉ dùng để phân tích. Khi $\lambda = 0$, C trùng hoàn toàn với B. λ nhỏ (0,05) chỉ đảo thứ tự các ứng viên gần bằng điểm; λ lớn (0,5) gần như đưa mọi ứng viên trùng prior lên trước. Nếu không có prior, ví dụ khi LLM trả lỗi, C dùng thứ tự của B. Nhóm chọn trước $\lambda = 0{,}1$: trong top-30, độ chênh độ tương đồng giữa ứng viên đầu và ứng viên thứ 30 có trung vị 0,075, nên $\lambda = 0{,}1$ đủ đưa phần lớn ứng viên trùng prior lên đầu nhưng chưa hoàn toàn là lọc cứng.

4.5 Prompt và cách đọc câu trả lời

Phần system định nghĩa sáu nhãn, yêu cầu trả về đúng một JSON có khóa label và nhắc rằng nội dung request là dữ liệu không tin cậy, không được làm theo lệnh

nằm trong đó, để chống prompt injection từ payload. Phần user là một JSON gồm danh sách ví dụ và request cần phân loại; ví dụ được xếp hạng cao nhất đặt cuối, sát request. Request cần phân loại không bị cắt; mỗi ví dụ bị cắt ở 500 ký tự và có đánh dấu. Câu trả lời sai định dạng được ghi là invalid_output và tính là sai, không bị loại khỏi mẫu số.

```
{"labeled_examples": [
   {"request": "{\"method\": \"GET\", \"uri\": \"/?p=1+AND+1=1\", ...}", "label":
"SQL_INJECTION"},
   ...],
 "request_to_classify": "{\"method\": \"GET\", \"uri\": \"/wp-json/...\", ...}"}
=> LLM trả về: {"label": "SQL_INJECTION"}
```

4.6 Quy trình đánh giá

Tham số k, λ và nguồn prior chỉ được chọn trên tập validation. Sau khi chọn, nhóm ghi lại cấu hình và mã commit, rồi mới chạy tập test (300 request đầu của tập con test, 50 mỗi lớp) một lần duy nhất. Ngoài Macro-F1, đề tài báo cáo accuracy, F1 từng lớp, FPR trên lớp Normal, LCR@k, số lượt gọi LLM, số token và độ trễ trên mỗi request. Chi phí của C tính cả lượt gọi tạo prior. Với RQ3, các request được tách theo prior đúng hay sai, rồi đếm số lỗi của B mà C sửa được và số câu B đúng mà C làm sai.

CHƯƠNG 5. CÀI ĐẶT VÀ KIỂM THỬ

5.1 Cấu trúc mã nguồn

Mã nguồn viết bằng Python, chia thành các module nhỏ có chú thích tiếng Việt (Bảng 5.1).

Bảng 5.1: Các module chính

Module	Chức năng
src/preprocessing/	Tiền xử lý, chia tập, kiểm tra rò rỉ, khảo sát dữ liệu
src/retrieval/views.py, embedding.py, index.py	Cách trình bày request, embedding, tìm top-N
src/retrieval/knn.py, selectors.py	kNN vote; các chiến lược A1, B, BAL, C
src/experiments/build_retrieval.py	Dựng vector pool và top-N cho mỗi query
src/experiments/eval_retrieval.py	Đánh giá RQ1, kNN, LCR@k (không cần LLM)
src/experiments/prompting.py, run_llm.py	Prompt, gọi LLM, lưu kết quả, chạy tiếp khi gián đoạn
src/analysis/metrics.py, compare.py	Chỉ số, bootstrap, McNemar, bảng và hình so sánh

Môi trường: Python 3.11.9 trên Windows 11, CPU 8 nhân; numpy, sentence-transformers 5.1.0. LLM gọi qua API tương thích OpenAI (Groq, mô hình openai/gpt-oss-20b, temperature 0, reasoning_effort low); khóa API đặt trong file .env, không đưa vào mã nguồn hay cấu hình. Mã hóa 3.000 request của pool mất khoảng 5,6 phút trên CPU.

5.2 Kiểm thử

Bộ kiểm thử gồm 55 bài, đều đạt trên Windows 11, trong đó có kiểm thử từ đầu đến cuối cho mọi pipeline bằng một LLM giả (trả về nhãn chiếm đa số trong các ví dụ). Bảng 5.2 nêu các tính chất quan trọng được kiểm tra.

Bảng 5.2: Các tính chất được kiểm thử tự động

Tính chất	Ý nghĩa
Nhãn và metadata của query không lọt vào prompt	Không rò rỉ đáp án sang LLM
Ví dụ luôn lấy từ pool, không phải từ tập đánh giá	Không dùng chính query làm ví dụ
$\lambda = 0$ cho kết quả trùng B; thiếu prior thì C dùng thứ tự của B	Công thức C cài đặt đúng
Tập test bị khóa nếu không có cờ --allow-test	Không vô tình dùng test để chỉnh tham số
Đổi cấu hình, dữ liệu hoặc mã thì phải dùng thư mục mới	Kết quả luôn khớp với cấu hình
Lỗi API được ghi lại và tính là sai	Không âm thầm loại các dự đoán thất bại
Chạy lại cùng lệnh thì tiếp tục từ chỗ dừng	Không mất kết quả khi mất mạng hoặc hết quota

5.3 Các sự cố đã xử lý

Bảng 5.3: Sự cố kỹ thuật và cách xử lý

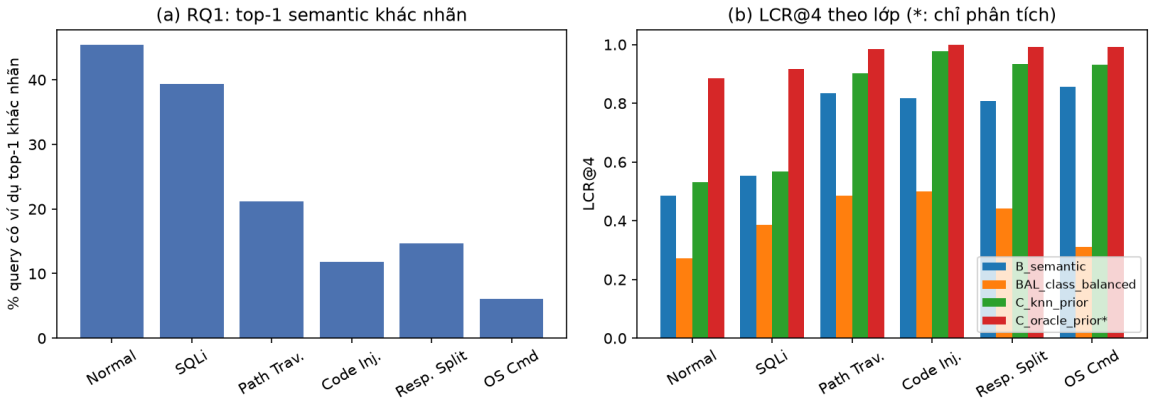
Sự cố	Nguyên nhân	Cách xử lý
API Groq trả HTTP 403	Cloudflare chặn User-Agent mặc định Python-urllib (error code 1010). Đã kiểm chứng: không có khóa, UA mặc định nhận 403, UA tùy chỉnh nhận 401	Thêm header User-Agent riêng
Không chạy được trên Windows	Dùng fcntl chỉ có trên Linux; file SQLite tạm chưa đóng khi lỗi; đọc file không chỉ định UTF-8	Khóa thư mục đa nền tảng (msvcrt), đóng file trước khi dọn, thêm encoding UTF-8
Giới hạn quota API miễn phí	Khoảng 8.000 token/phút và 200.000 token/ngày cho mỗi khóa	Tự chờ theo retry-after; hết quota ngày thì dừng gọn, chạy tiếp hôm sau; luân phiên khóa của hai thành viên

CHƯƠNG 6. KẾT QUẢ GIỮA KỲ

Mục 6.1 đến 6.3 không cần LLM; Mục 6.4 là kết quả LLM thật đầu tiên (A0). Tất cả tính trên 200 request của tập validation, với pool 3.000 ví dụ, N = 30, k = 4, λ = 0,1. Mọi số liệu được đọc từ các file kết quả trong thư mục src/artifacts của repository (danh sách lệnh ở Phụ lục B).

6.1 RQ1: ví dụ giống nhất có khác nhãn không

Có **23,0%** request mà ví dụ giống nhất trong pool mang nhãn khác, dù độ tương đồng trung bình của ví dụ đó rất cao (0,956). Hiện tượng tập trung ở lớp Normal (45,5%) và SQL Injection (39,4%), là hai lớp có payload ngắn chèn vào URI bình thường; các lớp có dấu hiệu rõ như OS Command Injection chỉ 6,1% (Hình 6.1a). Kết quả xác nhận động cơ của đề tài: với gần một phần tư request, semantic retrieval đưa cho LLM ví dụ gần nhất mang nhãn khác.



Hình 6.1: (a) Tỷ lệ top-1 khác nhãn theo lớp; (b) LCR@4 theo lớp của các chiến lược chọn ví dụ

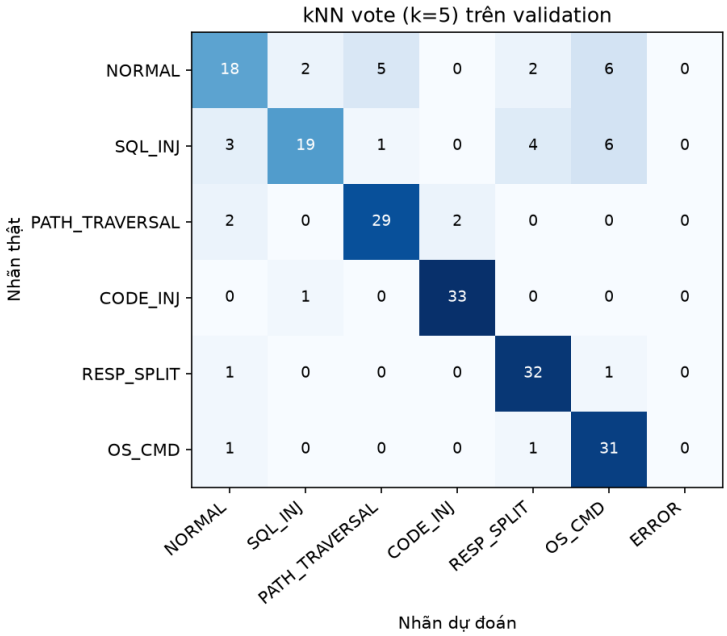
6.2 Đối chứng kNN không dùng LLM

Bảng 6.1 cho thấy kNN vote đạt Macro-F1 khoảng 0,80 và ổn định khi k từ 3 đến 10. Như vậy bài toán không tầm thường với phương pháp chỉ dựa trên độ tương đồng, và LLM có dư địa để cải thiện. Điểm yếu lớn nhất là FPR trên lớp Normal: khoảng 45% request Normal bị đoán là tấn công. Ma trận nhầm lẫn (Hình 6.2) cho thấy Normal và SQL Injection hay bị nhầm sang OS Command Injection và HTTP

Response Splitting, phù hợp với việc các payload kiểu sleep hay ký tự xuống dòng xuất hiện ở nhiều lớp.

Bảng 6.1: Kết quả kNN vote trên validation (200 request)

k	Macro-F1	Accuracy	FPR Normal
1	0,768	0,770	45,5%
3	0,805	0,815	42,4%
5	0,800	0,810	45,5%
7	0,801	0,810	42,4%
10	0,800	0,810	45,5%



Hình 6.2: Ma trận nhầm lẫn của kNN vote (k = 5) trên validation

Trên tập development (30 request), Macro-F1 của kNN với k = 5 là 0,760, nhưng tỷ lệ top-1 khác nhãn chỉ 6,7%, thấp hơn nhiều so với validation. Với 30 request (5 mỗi lớp), mỗi request chiếm hơn 3 điểm phần trăm, nên con số trên development dao động lớn và chỉ dùng để chạy thử; kết luận giữa kỳ dựa trên validation.

6.3 Ảnh hưởng của prior lên các ví dụ được chọn

Bảng 6.2 so sánh LCR@4 của bốn cách chọn ví dụ, với λ = 0,1. Prior kNN đúng 81,0% số request.

Bảng 6.2: LCR@4 của các chiến lược chọn ví dụ trên validation

Chiến lược	LCR@4 trung bình	Request không có ví dụ cùng nhãn
B (semantic top-k)	0,726	7,0%
BAL (cân bằng lớp)	0,400	0,0%
C, prior kNN	0,809	16,5%
C-oracle (chỉ phân tích)	0,963	0,0%

Ba nhận xét. Thứ nhất, xếp lại theo prior kNN nâng LCR từ 0,726 lên 0,809, và mức trần khi biết nhãn thật là 0,963, cho thấy pool thường có ví dụ đúng lớp trong top-30 nhưng không nằm ở đầu danh sách. Thứ hai, cái giá của C là khi prior sai: tỷ lệ request không còn ví dụ đúng lớp nào tăng từ 7,0% ở B lên 16,5% ở C. Đây là rủi ro tự xác nhận mà RQ3 cần đo bằng LLM thật: với những request này, LLM chỉ thấy ví dụ của lớp sai. Thứ ba, BAL có LCR thấp nhất (0,400) nhưng không request nào thiếu ví dụ đúng lớp, nên là đối chứng quan trọng để tách lợi ích của việc có đủ các lớp khỏi lợi ích của việc có nhiều ví dụ cùng lớp.

Để chọn λ mà không tốn lượt gọi LLM, nhóm quét λ trên chính phần truy hồi (Bảng 6.3). λ càng lớn thì tỷ lệ ví dụ cùng nhãn càng cao, nhưng số request mất hết ví dụ đúng lớp cũng tăng; từ $\lambda = 0{,}25$ trở lên kết quả không đổi vì mọi ứng viên trùng prior đã được đưa lên đầu. Giá trị $\lambda = 0{,}1$ nằm giữa hai thái cực này.

Bảng 6.3: Quét λ cho C với prior kNN (k = 4, validation)

λ	LCR@4 trung bình	Request không có ví dụ cùng nhãn
0,00	0,726	7,0%
0,02	0,779	10,5%
0,05	0,795	14,0%
0,10	0,809	16,5%
0,25	0,814	17,5%
0,50	0,814	17,5%
1,00	0,814	17,5%

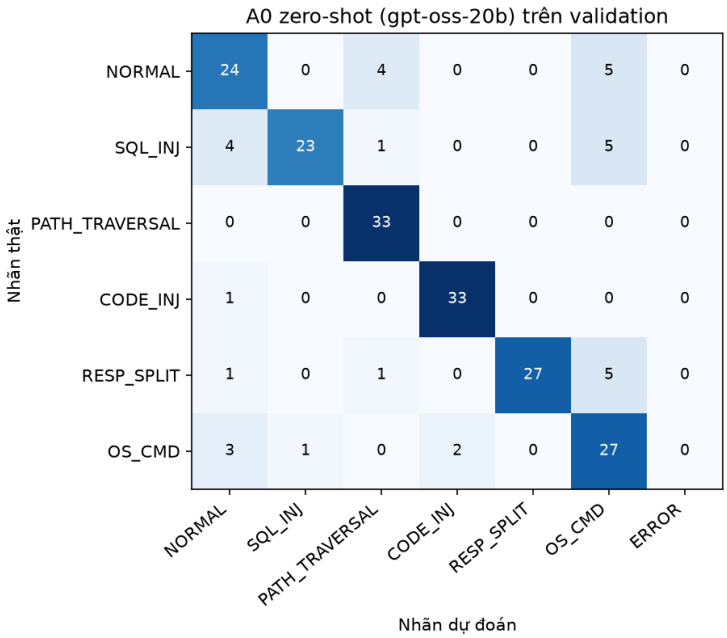
6.4 Kết quả LLM zero-shot (A0)

A0 dùng openai/gpt-oss-20b qua Groq, không có ví dụ trong prompt. Cả 200 request đều nhận được câu trả lời hợp lệ (coverage 100%), trung bình 637 token và 2,9 giây mỗi request. Bảng 6.4 so sánh A0 với kNN vote trên cùng 200 request.

Bảng 6.4: So sánh LLM zero-shot (A0) với kNN vote (k = 5) trên validation

Lớp	F1 của A0	F1 của kNN
Normal	0,73	0,62
SQL Injection	0,81	0,69
Path Traversal	0,92	0,85
Code Injection	0,96	0,96
HTTP Response Splitting	0,89	0,88
OS Command Injection	0,72	0,81
Macro-F1	**0,835**	0,800
FPR Normal	27,3%	45,5%

A0 cao hơn kNN 0,035 điểm Macro-F1, với khoảng tin cậy 95% bootstrap [-0,031; 0,104] và p McNemar = 0,542 (24 request chỉ A0 đúng, 19 request chỉ kNN đúng). Vì khoảng tin cậy vẫn chứa 0, chênh lệch này chưa đủ để khẳng định LLM tốt hơn kNN với 200 request. Điểm mạnh rõ nhất của A0 là lớp Normal: FPR giảm từ 45,5% xuống 27,3%. Ngược lại, A0 hay đoán nhầm sang OS Command Injection: ma trận nhầm lẫn (Hình 6.3) cho thấy nhiều request Normal, SQL Injection và HTTP Response Splitting bị đoán thành lớp này, có thể do các payload kiểu sleep hay ký tự xuống dòng mà định nghĩa nhãn trong prompt chưa phân biệt rõ. Đây là chỗ các ví dụ trong prompt (B, BAL, C) có thể giúp.



Hình 6.3: Ma trận nhầm lẫn của A0 trên validation

6.5 Giới hạn quota và điều chỉnh quy mô thí nghiệm

Khi chạy thật, nhóm đo được giới hạn của gói Groq miễn phí cho gpt-oss-20b: khoảng 8.000 token mỗi phút và 200.000 token mỗi ngày cho mỗi khóa API. Với 6 ví dụ cắt ở 1.500 ký tự, mỗi request có ví dụ tốn khoảng 2.600 token, nên một pipeline trên 200 request validation cần hơn hai ngày quota. Nhóm dừng lần chạy thử B với cấu hình này sau 34 request và không dùng kết quả đó. Để vừa quota mà vẫn giữ đủ các đối chứng chính, nhóm điều chỉnh trước khi chạy các pipeline có ví dụ:

- k = 4 ví dụ, mỗi ví dụ cắt ở 500 ký tự: khoảng 1.300 token mỗi request (A0 không bị ảnh hưởng vì không có ví dụ).
- Chỉ chạy bộ chính A0, B, BAL, C prior kNN, C prior LLM với $\lambda = 0{,}1$ cố định; ảnh hưởng của λ được phân tích bằng bảng quét λ ở Mục 6.3; C-oracle và A1 chỉ chạy nếu còn quota.
- Dùng khóa API của cả hai thành viên, luân phiên tự động khi một khóa hết quota ngày.
- Tập test dùng 300 request đầu của tập con test (50 mỗi lớp), cố định trước khi xem kết quả.

Với hai khóa, bộ chính trên validation cần khoảng 3 ngày quota và tập test khoảng 4,5 ngày.

CHƯƠNG 7. KẾT LUẬN GIỮA KỲ VÀ KẾ HOẠCH

7.1 Kết quả đạt được

- Xử lý xong SR-BH 2020: 466.267 request duy nhất, chia tập theo nhóm, không rò rỉ giữa các tập, tái lập được trên Linux và Windows.
- Cài đặt toàn bộ hệ thống: truy hồi, kNN, năm chiến lược chọn ví dụ, chạy LLM có khả năng chạy tiếp, so sánh có kiểm định; 55 bài kiểm thử tự động đều đạt.
- Trả lời RQ1 bằng số liệu thật: 23,0% request có ví dụ giống nhất mang nhãn khác.
- Có đối chứng kNN (Macro-F1 0,800) và kết quả LLM zero-shot thật (Macro-F1 0,835), cùng bằng chứng sơ bộ cho cả lợi ích lẫn rủi ro của C.

7.2 Hạn chế

- Chỉ một bộ dữ liệu, là traffic của một website WordPress trong 12 ngày.
- Nhãn có dấu hiệu nhiễu; điểm số đo mức đồng thuận với nhãn nguồn.
- Có endpoint chung giữa pool và tập đánh giá; chưa đo khả năng tổng quát sang endpoint mới.
- Tập con cân bằng lớp, không phản ánh tỷ lệ traffic thật; tập validation chỉ 200 request.
- Mỗi request tốn một đến hai lượt gọi LLM, chỉ phù hợp phân tích log offline.
- Mới có kết quả LLM của A0; quy mô thí nghiệm bị giới hạn bởi quota API miễn phí (k = 4, test 300 request).

7.3 Kế hoạch giai đoạn cuối kỳ

Bảng 7.1: Kế hoạch đến khi nộp báo cáo cuối kỳ

Thời gian	Công việc
20–22/10	Chạy B, BAL, C prior kNN, C prior LLM trên validation bằng hai khóa API
22–25/10	Hai thành viên đọc lại độc lập 60 mẫu nhãn; chạy C-oracle nếu còn quota
26–27/10	So sánh, bootstrap, McNemar, bảng prior đúng/sai trên validation
28/10	Khóa cấu hình (ghi commit, config, model)
29/10–04/11	Chạy tập test 300 request một lần cho bộ chính; phân tích lỗi theo lớp
09–14/11	Hoàn thiện báo cáo cuối kỳ, kiểm tra tái lập, nộp

Ngân sách ước tính khoảng 2,9 triệu token cho bộ chính (validation và test), tương đương khoảng 7,5 ngày quota của hai khóa. Nếu C không tốt hơn B, đó vẫn là kết quả có giá trị: nhóm sẽ dùng bảng prior đúng/sai, bảng quét λ và C-oracle (nếu có) để giải thích nguyên nhân.

TÀI LIỆU THAM KHẢO

[1] Liu, J., Shen, D., Zhang, Y., Dolan, B., Carin, L., và Chen, W. (2022). What Makes Good In-Context Examples for GPT-3? Proceedings of DeeLIO 2022: Deep Learning Inside Out, 100–114.

[2] Rubin, O., Herzig, J., và Berant, J. (2022). Learning To Retrieve Prompts for In-Context Learning. Proceedings of NAACL 2022, 2655–2671.

[3] Lin et al. (2025). CLL-RetICL: Contrastive Linguistic Label Retrieval-based In-Context Learning for Text Classification via Large Language Models. Findings of IJCNLP 2025. (Cần đối chiếu số trang trước bản cuối.)

[4] In-Context Learning with Noisy Labels (2024). arXiv:2411.19581.

[5] DemoShapley: Valuation of Demonstrations for In-Context Learning (2024). arXiv:2410.07523.

[6] Blefari, F. et al. (2025). CyberRAG: An Agentic RAG Cyber Attack Classification and Reporting Tool. arXiv:2507.02424.

[7] Reimers, N., và Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP 2019, 3982–3992.

[8] Efron, B., và Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman & Hall.

[9] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157.

[10] SR-BH 2020 multi-label dataset, phiên bản 1.2 [Bộ dữ liệu]. Harvard Dataverse. https://doi.org/10.7910/DVN/OGOIXX. Giấy phép CC0 1.0.

PHỤ LỤC A. PHÂN CÔNG CÔNG VIỆC

Nhóm điền thông tin đúng với phần việc thực tế trước khi nộp. Tổng tỷ lệ đóng góp cần bằng 100%.

Bảng A.1: Phân công và mức độ hoàn thành

Họ tên	MSSV	Nội dung thực hiện	Đóng góp %	Hoàn thành %
		Dữ liệu, truy hồi, kNN, reranker C		
		API LLM, chạy pipeline, chỉ số, thống kê		

PHỤ LỤC B. LỆNH TÁI LẬP

Chạy từ thư mục gốc của repository. Hướng dẫn đầy đủ nằm ở reports/protocol/huong_dan_doc_va_chay_code.md.

```
pip install -r src/requirements-experiments.txt
python -m unittest discover -s src/tests
python -m src.preprocessing.prepare --config src/configs/data.json
python -m src.experiments.build_retrieval --output src/artifacts/retrieval/minilm
python -m src.experiments.eval_retrieval --retrieval src/artifacts/retrieval/minilm --split
validation
bash src/experiments/run_suite.sh validation   # A0, B, BAL, C; khóa trong .env, .env.tv2
python -m src.experiments.run_llm --config src/configs/experiments/B_k4.json \
    --llm src/configs/llm/groq.json --env-file .env --split validation \
    --output src/artifacts/runs/validation/B_k4
python -m src.analysis.compare --split validation --reference B_semantic_k4 --runs
"src/artifacts/runs/validation/*" \
    --output src/artifacts/reports/validation
```